

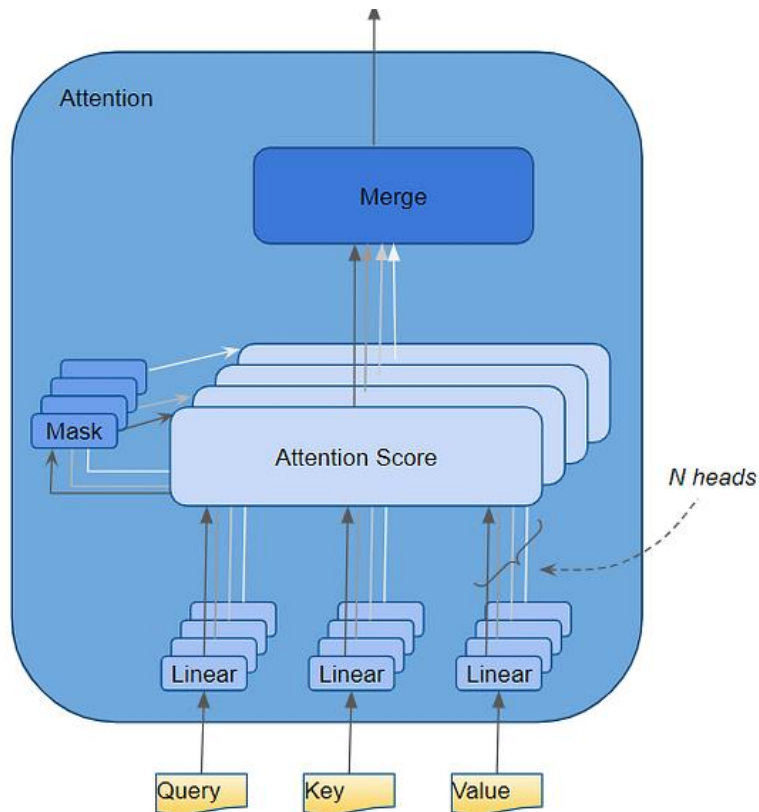


University
of Exeter

COMM117

Large Language Models and Application

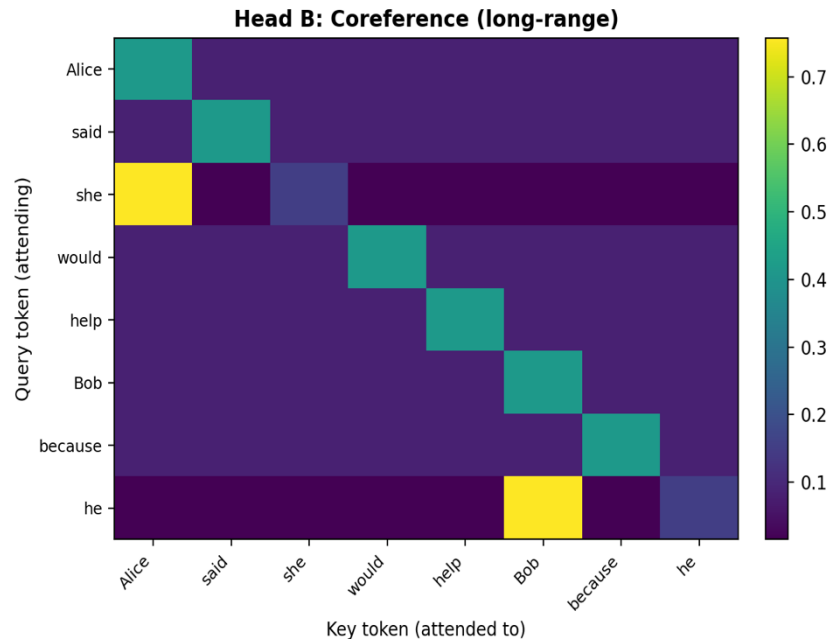
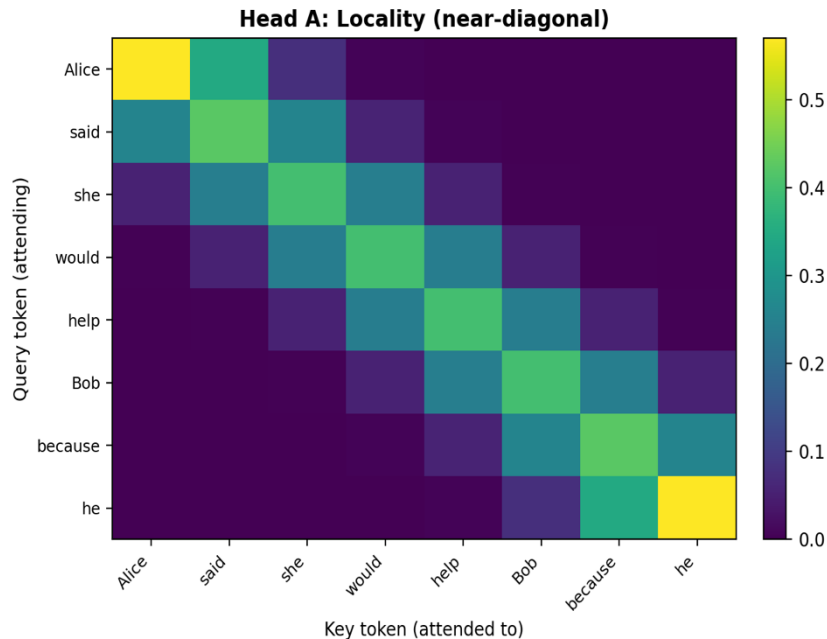
Previous Lecture



Different attention patterns

Same sentence, two heads learn different token-to-token alignments:

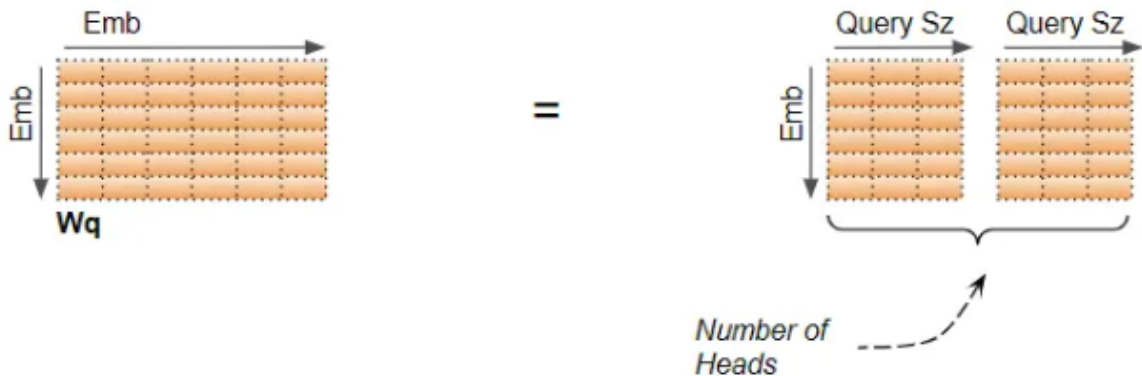
- Head A concentrates on near-diagonal (local phrase/syntax cues).
- Head B links pronouns to earlier antecedents (long-range coreference).



Multi-head Attention – Linear Layers

Implementation in code:

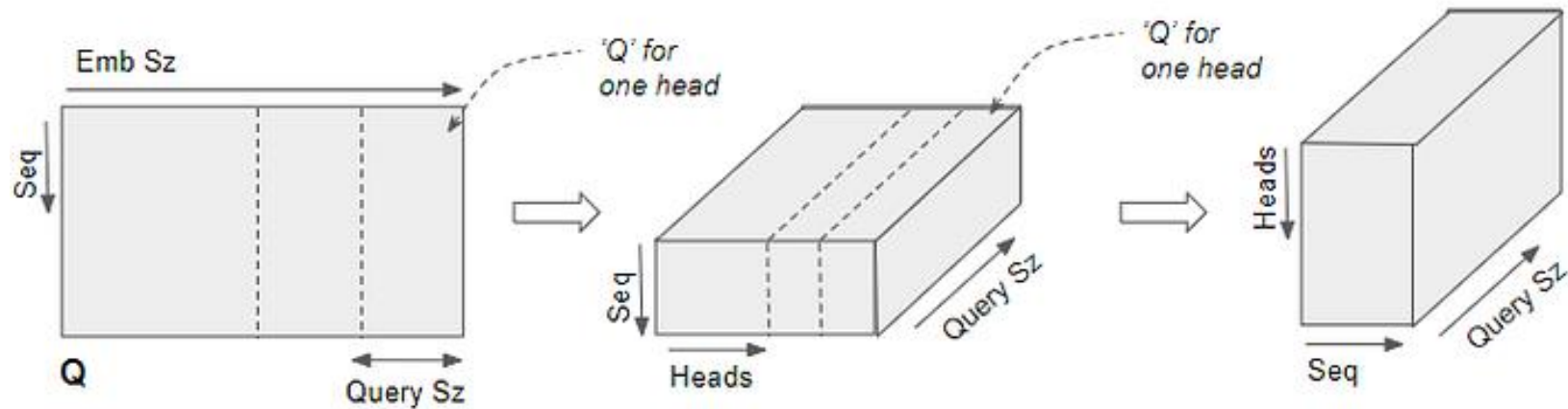
one bigger weight matrix that contains all heads' weights



Multi-head Attention – Linear Layers

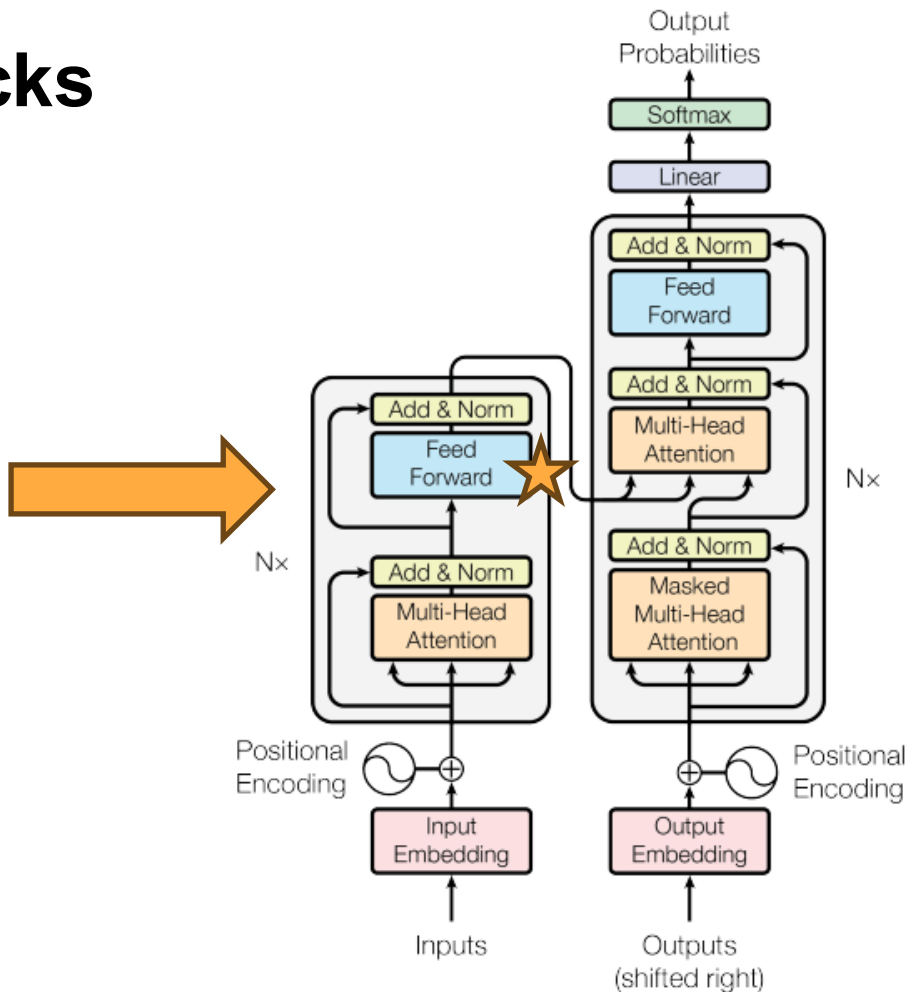
Split into heads!

- **Reshaping the Q, K, and V matrices**
- The Q, K, and V matrices output by the Linear layers are reshaped to include an explicit Head dimension.
- Each 'slice' corresponds to a matrix per head.



Transformer Blocks

- Attention mixes **information across tokens.**
- The FFN mixes **features within each token.**
- Residual+Norm keep **training stable**



Introduction Today

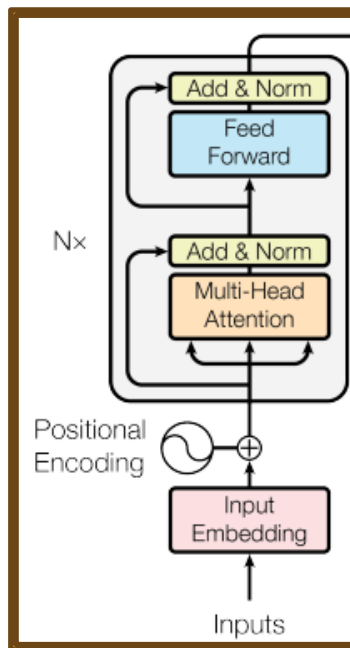
- Encoder – Decoder
- Cross attention
- Masking

Encoder-Decoder

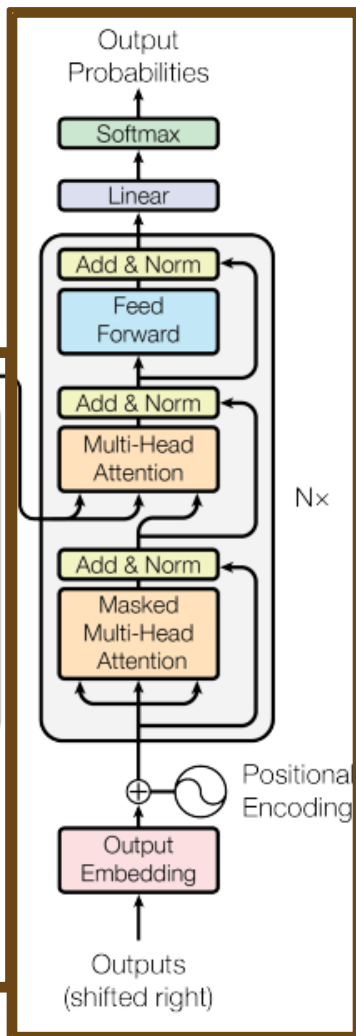


Encoder-Decoder

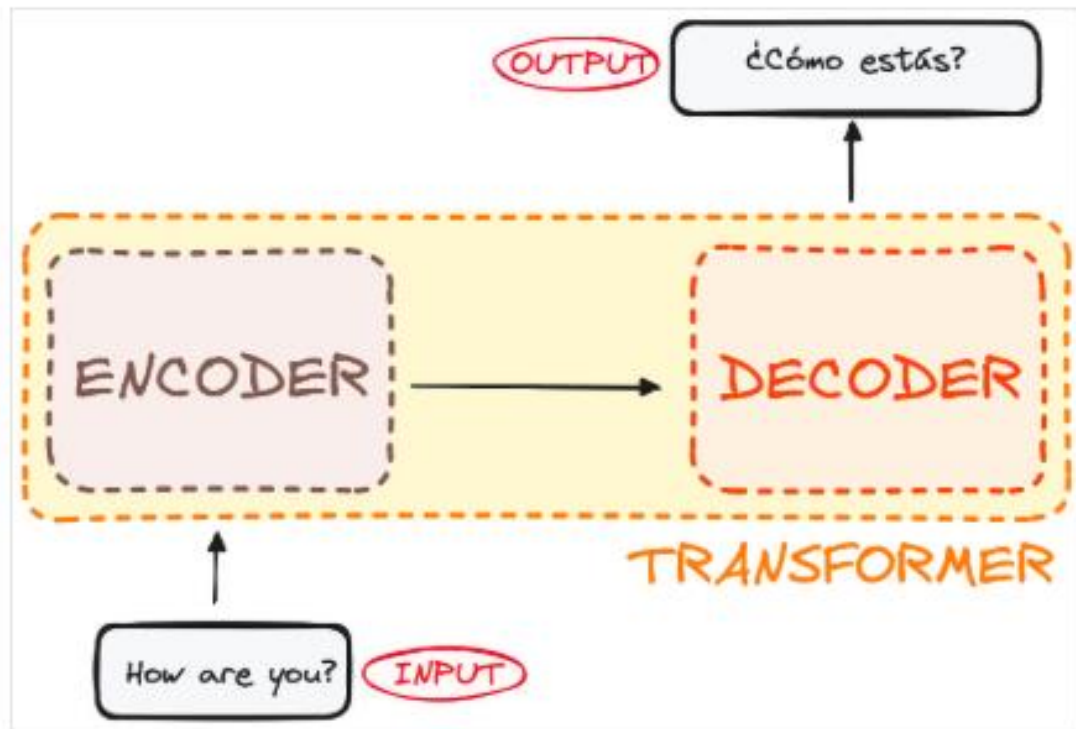
Encoder →



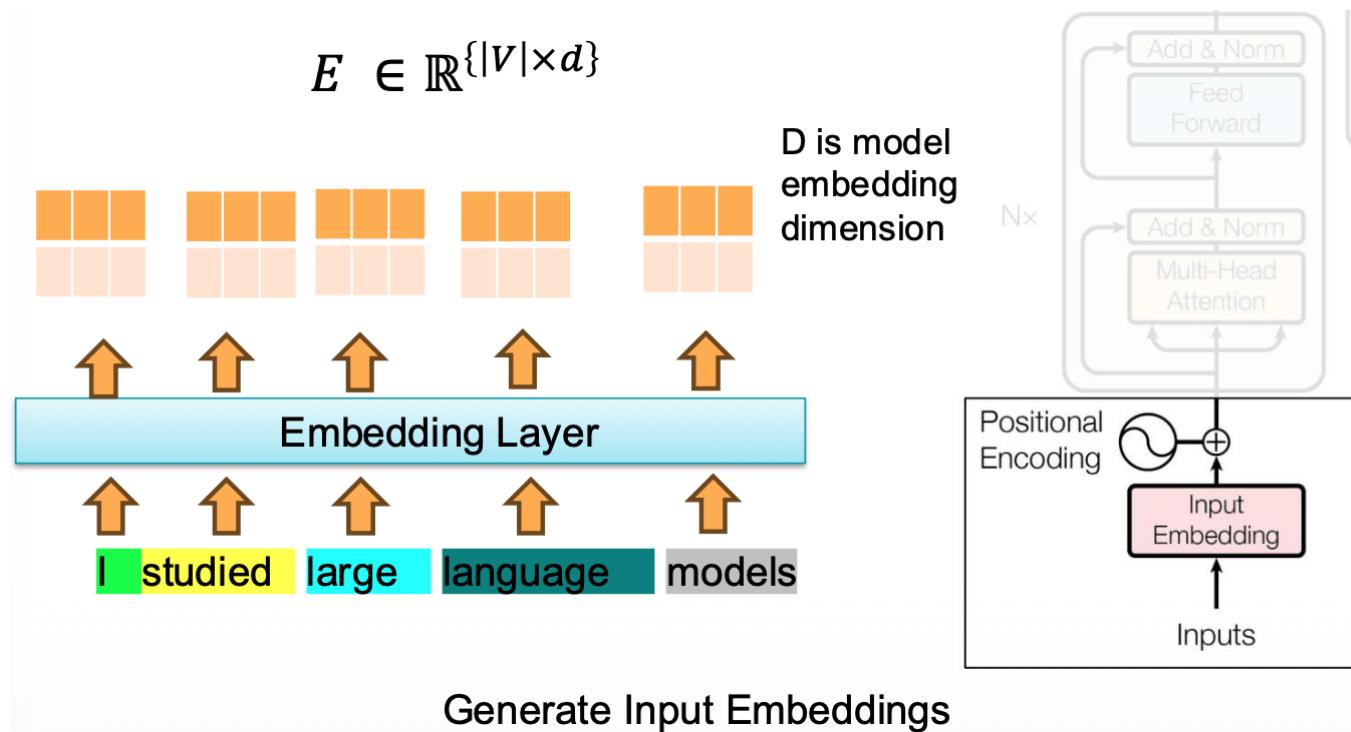
← **Decoder**



Encoder-Decoder

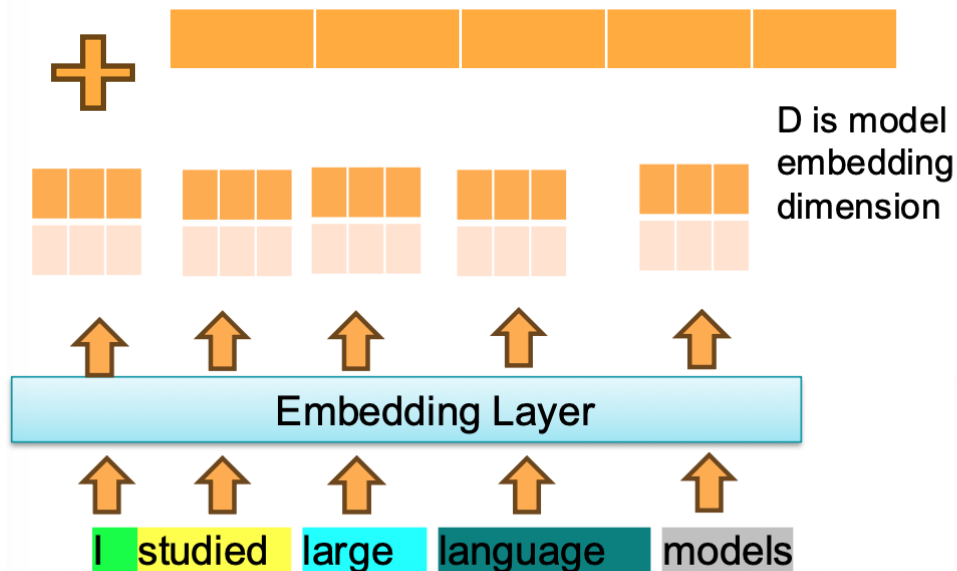


Positional Encoding

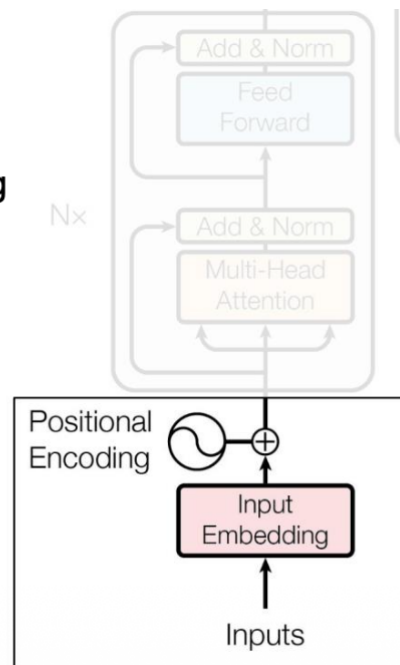


Positional Encoding

Positional Encoding

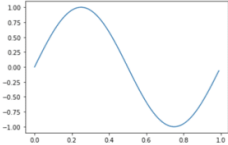
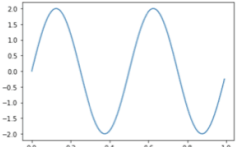
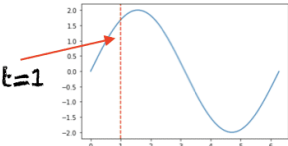


Position information
RNN Has Transformer Does not



Positional Encoding- How

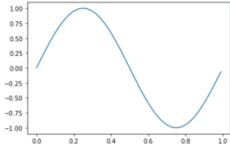
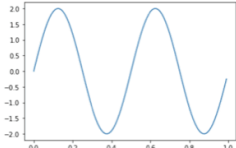
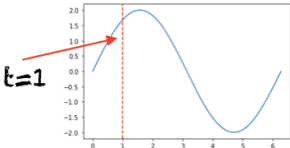
The only thing we change is **how fast it oscillates**
→ frequency

Equation	Graph	Frequency	Wavelength
$\sin(2\pi t)$		1	1
$\sin(2 * 2\pi t)$		2	1/2
$\sin(t)$		$1/2\pi$	2π
$\sin(ct)$	Depends on c	$c/2\pi$	$2\pi/c$

Give each position
[-1,1]

Positional Encoding- How

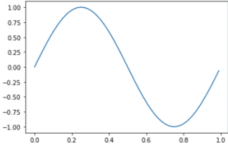
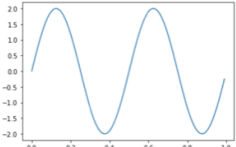
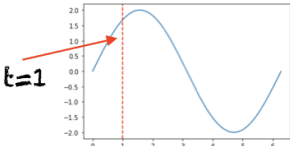
how quickly the curve goes up and down as you move along token positions.

Equation	Graph	Frequency	Wavelength
$\sin(2\pi t)$		1	1
$\sin(2 * 2\pi t)$		2	1/2
$\sin(t)$		$1/2\pi$	2π
$\sin(ct)$	Depends on c	$c/2\pi$	$2\pi/c$

High frequency:
→ distinguishes
nearby positions
well

Positional Encoding- How

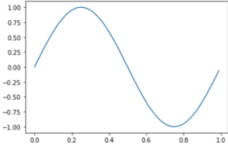
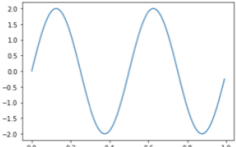
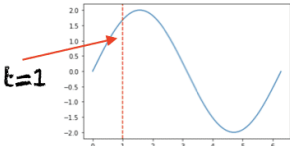
how quickly the curve goes up and down as you move along token positions.

Equation	Graph	Frequency	Wavelength
$\sin(2\pi t)$		1	1
$\sin(2 * 2\pi t)$		2	1/2
$\sin(t)$		$1/2\pi$	2π
$\sin(ct)$	Depends on c	$c/2\pi$	$2\pi/c$

Low frequency
→ captures
broader, long-
range position
trends.

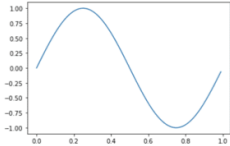
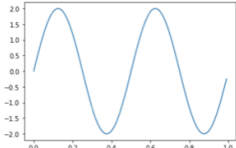
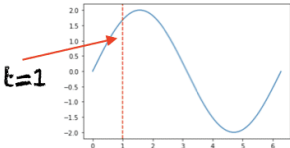
Positional Encoding- How

how quickly the curve goes up and down as you move along token positions.

Equation	Graph	Frequency	Wavelength	
$\sin(2\pi t)$		1	1	Medium frequency → “fills the gap” between local and global.
$\sin(2 * 2\pi t)$		2	1/2	
$\sin(t)$		$1/2\pi$	2π	
$\sin(ct)$	Depends on c	$c/2\pi$	$2\pi/c$	

Positional Encoding- How

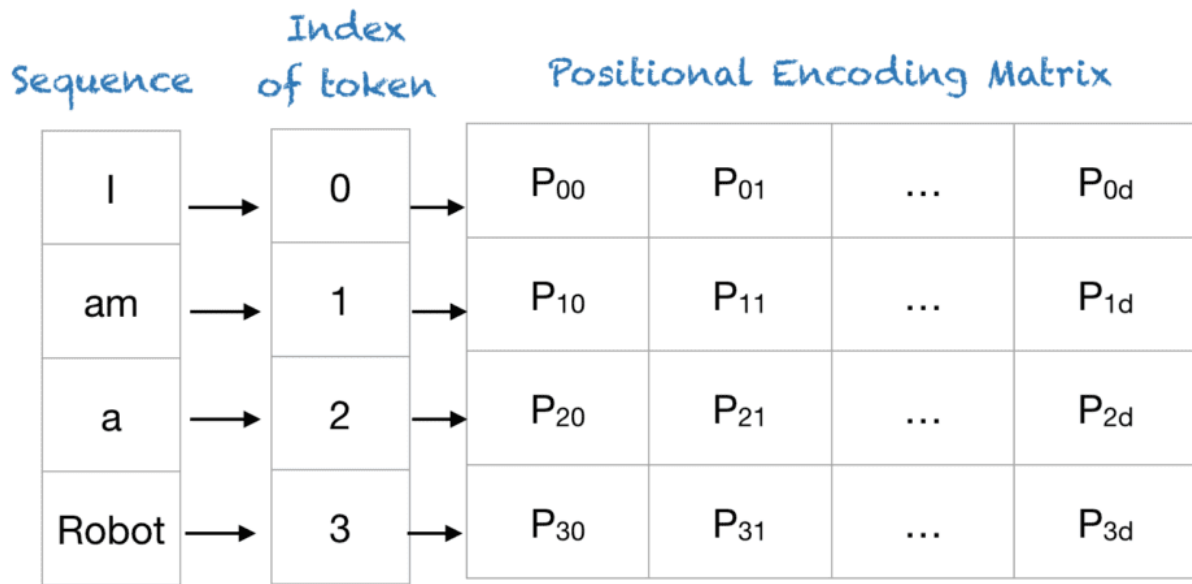
how quickly the curve goes up and down as you move along token positions.

Equation	Graph	Frequency	Wavelength
$\sin(2\pi t)$		1	1
$\sin(2 * 2\pi t)$		2	1/2
$\sin(t)$		$1/2\pi$	2π
$\sin(ct)$	Depends on c	$c/2\pi$	$2\pi/c$

By choosing many different c 's, we get multiple "position scales."

Positional Encoding- How

A Transformer uses **many sinusoids at different frequencies**, one per dimension.



Positional Encoding Matrix for the sequence 'I am a robot'

Positional Encoding- How

A Transformer uses **many sinusoids at different frequencies**, one per dimension.

position

$$PE(p, 2i) = \sin\left(\frac{p}{10000^{\frac{2i}{d}}}\right)$$

$$PE(p, 2i + 1) = \cos\left(\frac{p}{10000^{\frac{2i}{d}}}\right)$$

Different dimension different c

$\sin(ct)$

i: index over **frequency bands**
(0, 1, 2, ..., $d/2 - 1$)



$$PE(p) = [\sin(.), \cos(.), \sin(.), \cos(.), \dots]$$

i=0

i=1

i=2

i=d/2-1

Positional Encoding- How

➡ $PE(p) = [\sin(.), \cos(.), \sin(.), \cos(.), \dots]$

$i=0$ $i=1$ $i=2$ $i=d/2-1$

- Across the dimension:
 - some components vary quickly → encode fine position differences
 - some components vary slowly → encode long range position

Positional Encoding- How



PE

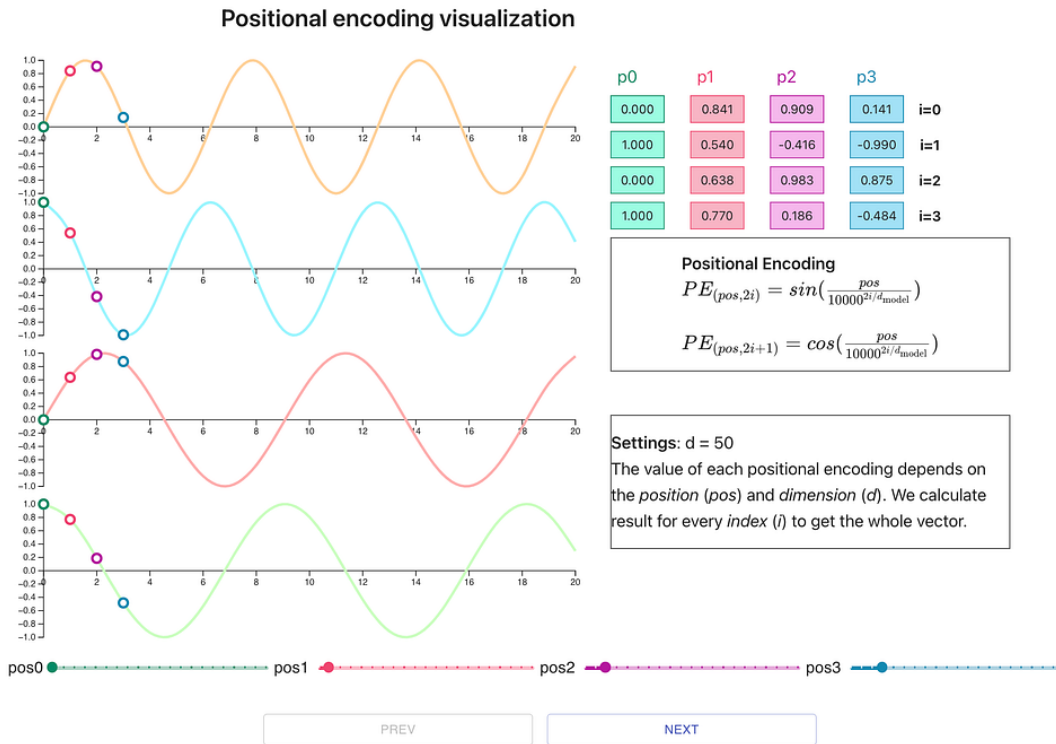


Figure 2: Positional Encoding visualization, Designed base on "Attention Is All You Need", NeurIPS 2017 Paper

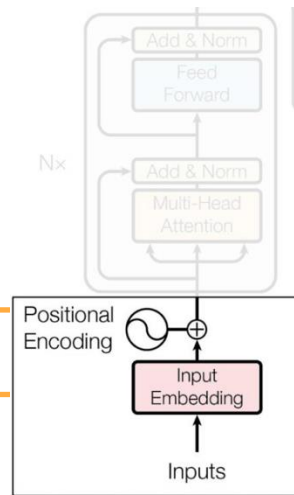
Morden Positional Encoding- LLM

➡ $PE(p) = [\sin(.), \cos(.), \sin(.), \cos(.), \dots]$ **Absolute PE**

token embeddings → (+ **absolute PE**) → QKV

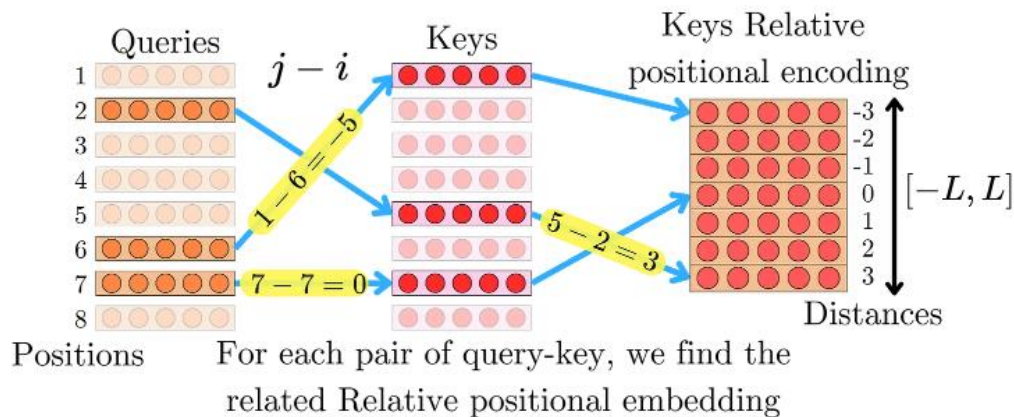


token embeddings → QKV → **Additive positional embedding** → scores



Morden Positional Encoding- Additive to K

$$\begin{aligned} \mathbf{k}_j &\rightarrow \mathbf{k}_j + \mathbf{a}_{ij}^K \\ \mathbf{v}_j &\rightarrow \mathbf{v}_j + \mathbf{a}_{ij}^V \end{aligned} \quad \Rightarrow \quad \text{Learned embedding from relative distance } j-i$$



Morden Positional Encoding- Additive to K

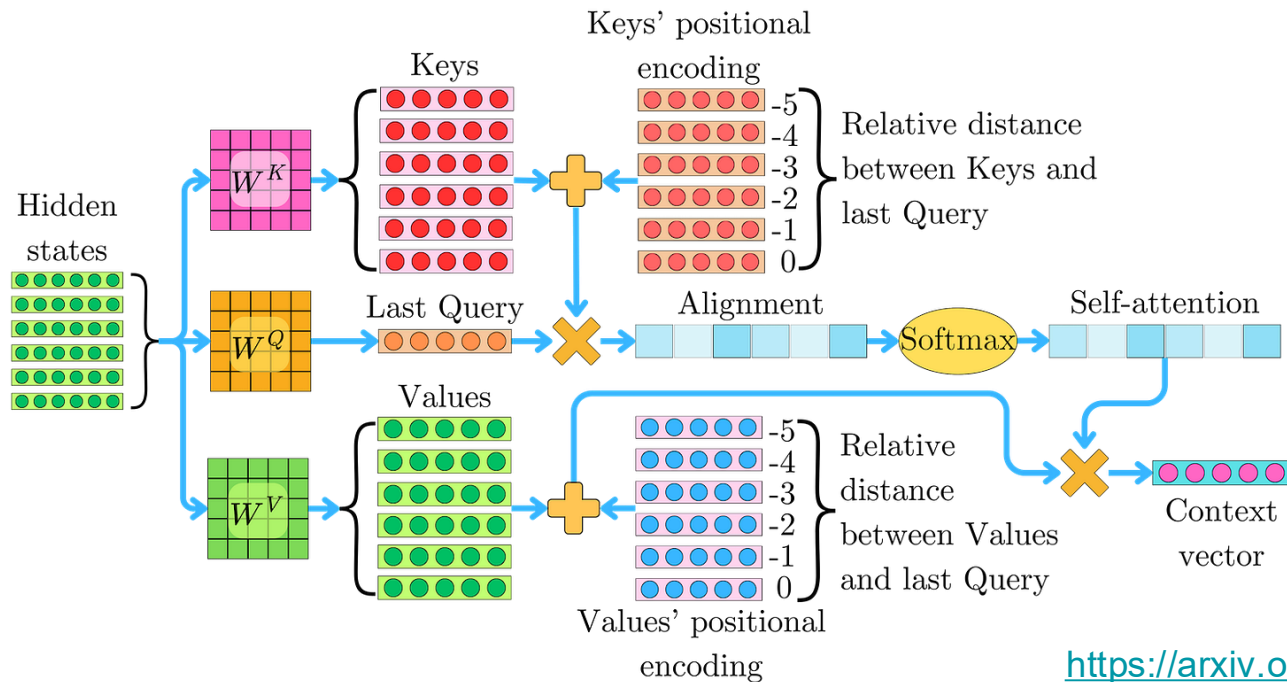
V

$$\mathbf{k}_j \rightarrow \mathbf{k}_j + \mathbf{a}_{ij}^K$$

$$\mathbf{v}_j \rightarrow \mathbf{v}_j + \mathbf{a}_{ij}^V$$



Learned embedding from relative distance j-i



Morden Positional Encoding- LLM

➡ $PE(p) = [\sin(.), \cos(.), \sin(.), \cos(.), \dots]$ **Absolute PE**

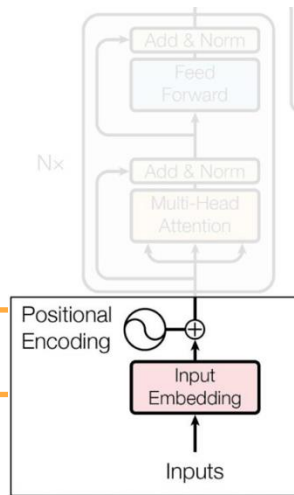
token embeddings $\rightarrow$ (+ **absolute PE**) $\rightarrow$ QKV



token embeddings $\rightarrow$ QKV $\rightarrow$ **Additive positional embedding K, V** $\rightarrow$ scores

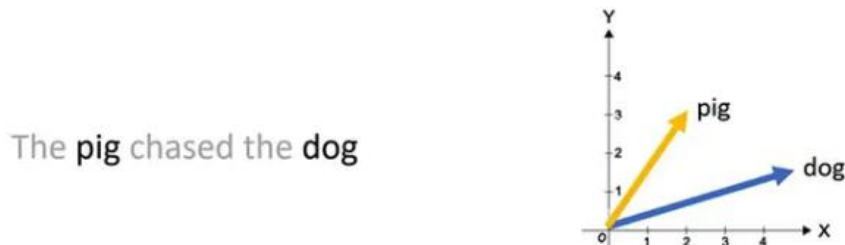


token embeddings $\rightarrow$ QKV $\rightarrow$ **apply RoPE to Q, K** $\rightarrow$ scores



Morden Positional Encoding- RoPE

- RoPE rotates Q and K by a position-dependent angle instead of adding positional vector.



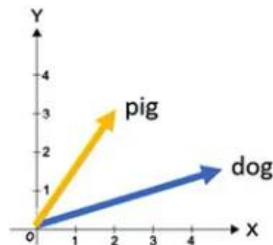
Pros:

- **Stability of Vectors:** Adding tokens at the end of a sentence doesn't affect the vectors for words at the beginning, facilitating efficient caching.
- **Preservation of Relative Positions:** If two words, say “pig” and “dog,” maintain the same relative distance in different contexts, their vectors are rotated by the same amount.

Morden Positional Encoding- RoPE

- RoPE rotates Q and K by a position-dependent angle instead of adding positional vector.

The pig chased the dog



$$f_{\{q,k\}}(\mathbf{x}_m, m) = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} W_{\{q,k\}}^{(11)} & W_{\{q,k\}}^{(12)} \\ W_{\{q,k\}}^{(21)} & W_{\{q,k\}}^{(22)} \end{pmatrix} \begin{pmatrix} x_m^{(1)} \\ x_m^{(2)} \end{pmatrix}$$

This induces **relative-position structure** in dot products.

Morden Positional Encoding- ALiBi

Attention With **L**inear **B**iases

Add distance-dependent bias to logits

Distance between the query q_i and k_j

$$e_{ij} = \mathbf{q}_i^\top \mathbf{k}_j + m_h(j - i)$$

$$\text{scores} = \frac{QK^\top}{\sqrt{d}} + b(\Delta)$$

Morden Positional Encoding

Position affects attention either by:

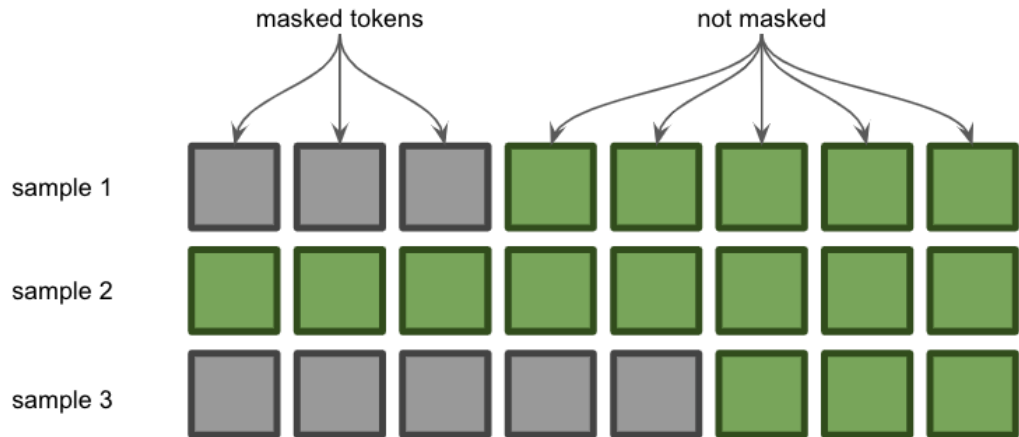
- Adding vectors to inputs (absolute PE)
- Rotating Q,K (RoPE)
- Adding a distance bias to logits (ALiBi-like)

Pick **ALiBi** when: you expect to **run much longer contexts** than you can afford to train on, and you want a **strong built-in long-range** extrapolation bias.

Pick **RoPE** when: you want a **strong general-purpose positional mechanism** that integrates into Q/K geometry and is **widely used in modern LLM stacks**; use RoPE scaling/continued training when extending context substantially.

Masking

- Positional encoding tells the model **where tokens are**
- Masking tells the model **which tokens it is allowed to look at**



Attention logits

$$\text{softmax} \left[S = \frac{QK^\top}{\sqrt{d_k}} + B_{\text{pos}} + M_{\text{mask}} \right]$$

Content-based similarity

Positional effect

(relative bias /
RoPE effect
can be viewed
as modifying
geometry)

Hard constraint

Position is a preference/structure; masks are permissions

Padding masking

$$|B| \begin{bmatrix} x_1 & x_2 & [\text{PAD}] & [\text{PAD}] & [\text{PAD}] & [\text{PAD}] \\ x_3 & x_4 & x_5 & x_6 & [\text{PAD}] & [\text{PAD}] \\ x_7 & x_8 & x_9 & [\text{PAD}] & [\text{PAD}] & [\text{PAD}] \\ x_{10} & x_{11} & x_{12} & x_{13} & x_{14} & [\text{PAD}] \end{bmatrix}$$

$\underbrace{\hspace{15em}}_{|L|}$

- In practice, to process different lengths together efficiently:

```
sequences = [
    ["The", "cat", "sat"], # Length 3
    ["Hello", "world"], # Length 2
    ["A", "quick", "brown", "fox"], # Length 4
]

# Pad to maximum length
max_len = max(len(seq) for seq in sequences)
pad_token = "[PAD]"

padded_sequences = []
for seq in sequences:
    padded = seq + [pad_token] * (max_len - len(seq))
    padded_sequences.append(padded)
```

```
Padded sequences (max length = 4):
Sequence 0: ['The', 'cat', 'sat', '[PAD]']
Sequence 1: ['Hello', 'world', '[PAD]', '[PAD]']
Sequence 2: ['A', 'quick', 'brown', 'fox']
```

$$S = \frac{QK^\top}{\sqrt{d_k}}, \quad S' = S + M_{\text{pad}}$$
$$(M_{\text{pad}})_{i,j} = \begin{cases} 0 & \text{if key token } j \text{ is real} \\ -\infty & \text{if key token } j \text{ is PAD} \end{cases}$$
$$A = \text{softmax}(S') \Rightarrow A_{i,\text{PAD}} \approx 0$$

Stops attention from putting weight on padding

Causal masking

Causal masking is required to match the next-token objective.

$$p(x_1, \dots, x_T) = \prod_{t=1}^T p(x_t \mid x_{<t})$$

```
def create_look_ahead_mask(size):  
    """Create look-ahead mask for autoregressive generation"""  
    mask = torch.triu(torch.ones(size, size), diagonal=1)  
    return mask == 0
```

If not masking future tokens:

- Letting the model access them during training would be **label leakage**

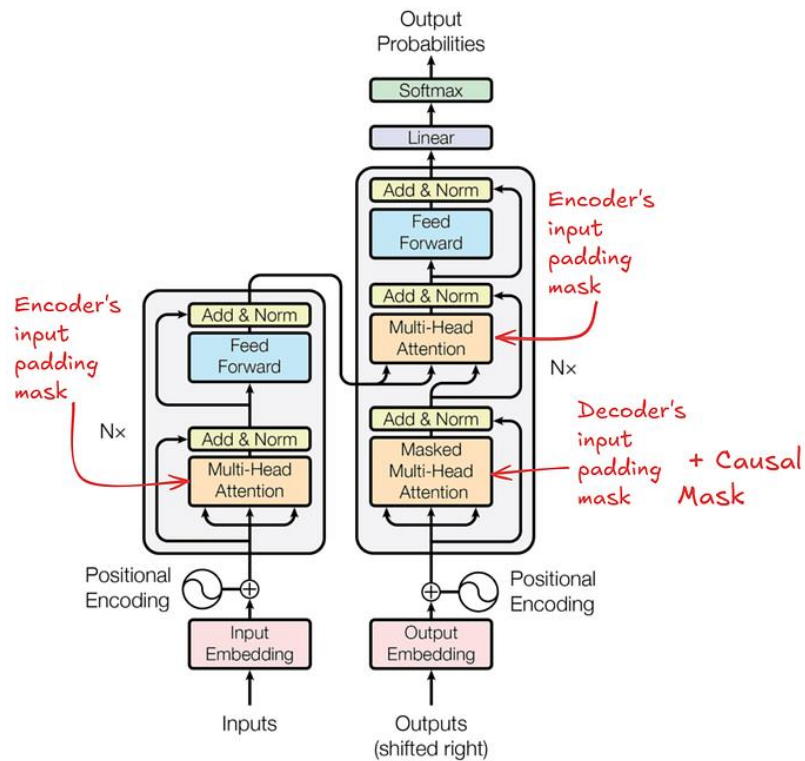
$$S = \frac{QK^T}{\sqrt{d_k}}, \quad S' = S + M_{\text{causal}}$$

$$(M_{\text{causal}})_{i,j} = \begin{cases} 0 & j \leq i \\ -\infty & j > i \end{cases}$$

$$A = \text{softmax}(S') \Rightarrow A_{i,j} = 0 \text{ for } j > i$$

	S1	S1	EOT	S2	S2
S1	1	0	0	0	0
S1	1	1	0	0	0
EOT	1	1	1	0	0
S2	1	1	1	1	0
S2	1	1	1	1	1

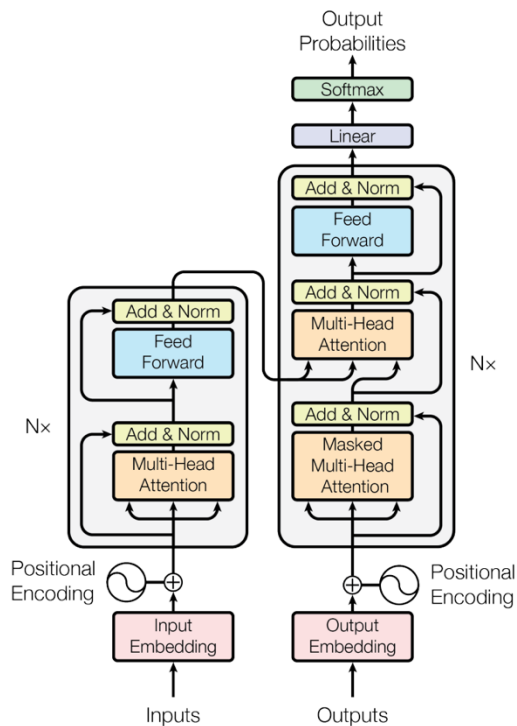
Masking



Encoder-Decoder

BERT

Encoder



GPT

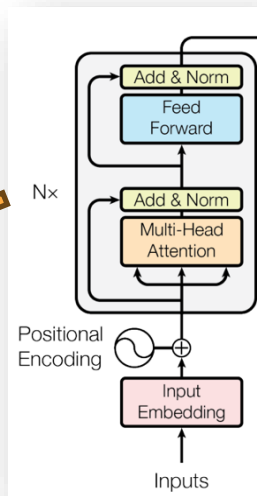
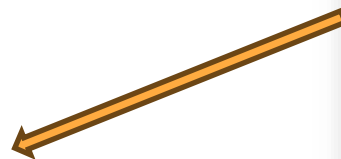
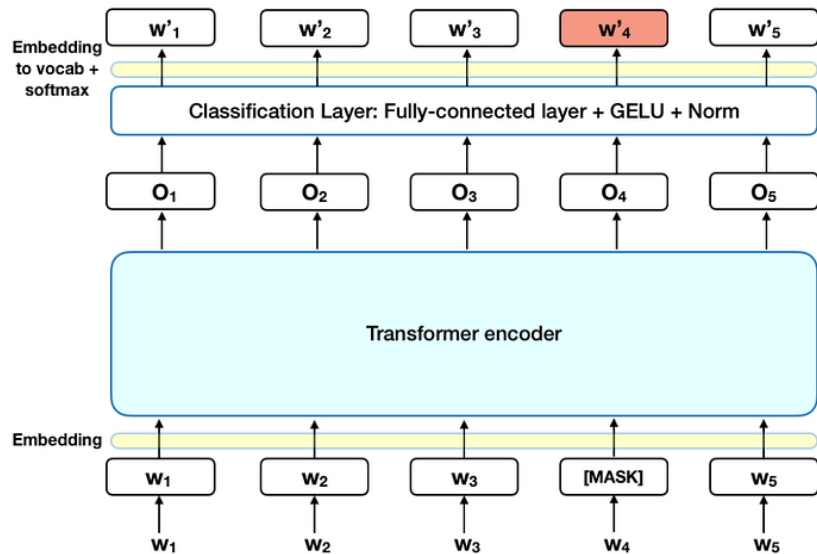
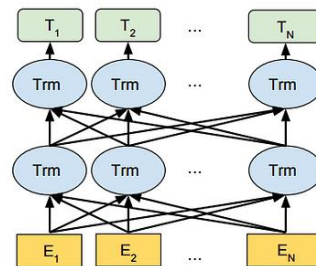
Decoder

Encoder-Decoder

- **Encoder-only models (like BERT):** Use a bidirectional approach to understand context from both directions → **bidirectional self-attn**
- **Decoder-only models (like GPT, Llama):** These models process text from left to right and are particularly good at text generation tasks → **causal self-attn**
- **Encoder-decoder models (like T5, BART):** These models combine both approaches, using an encoder to understand the input and a decoder to generate output. → **encoder self-attn + decoder masked self-attn + cross-attn**

BERT

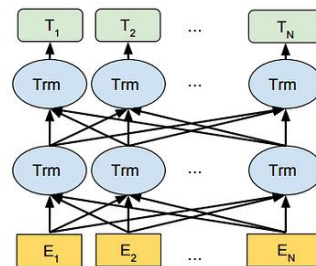
Bidirectional Encoder Representations from Transformers



BERT

Bidirectional Encoder Representations from Transformers

- **Padding masking only!– no causal masking**
It can use information from previous tokens and future tokens

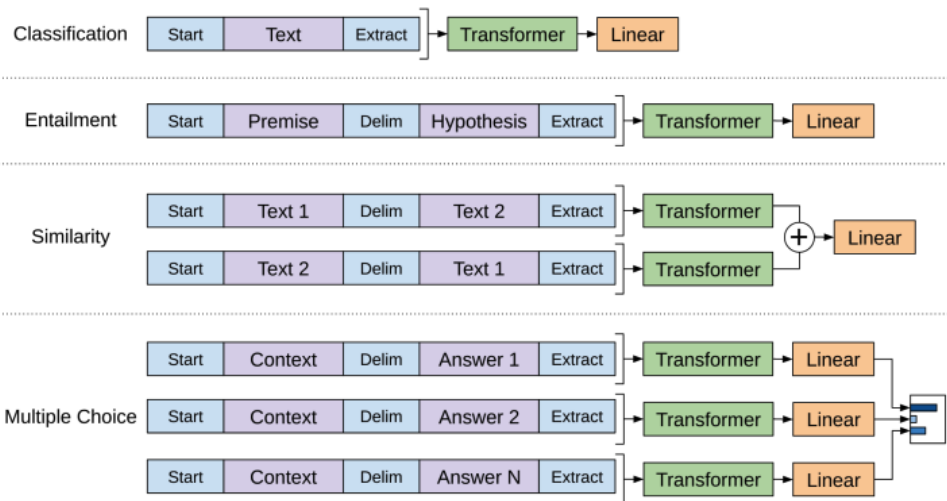
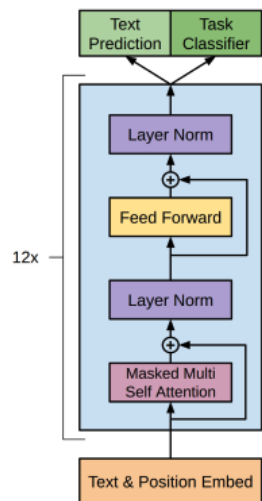


Encoder-only models are typically used when you want a **fixed representation of the input** (for **understanding tasks**):

- sentiment **classification**
- topic **classification**
- NER / token labeling
- retrieval embeddings

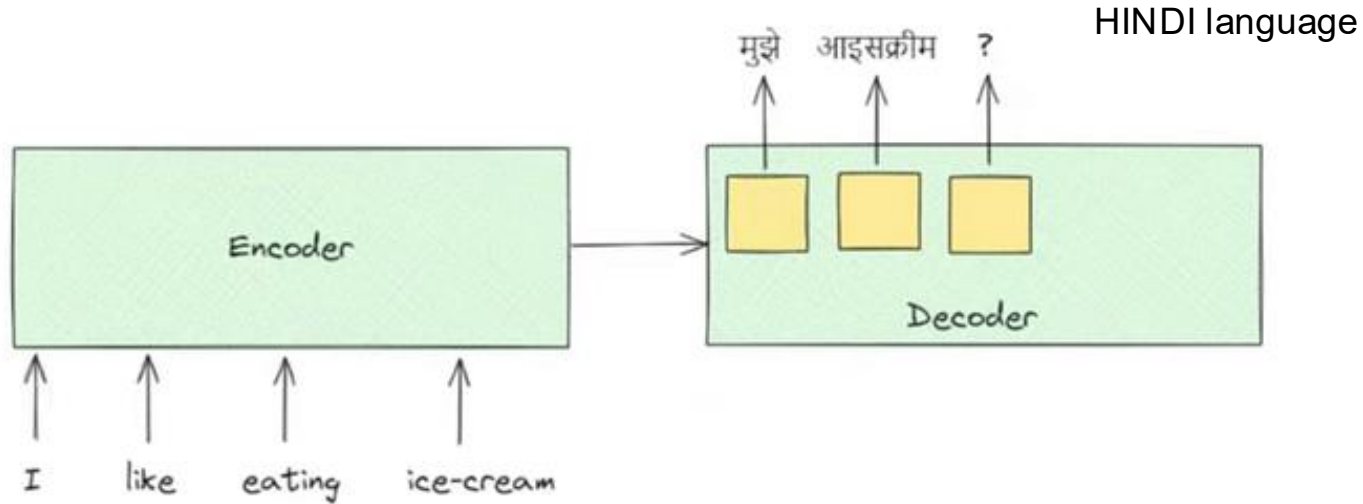


GPT

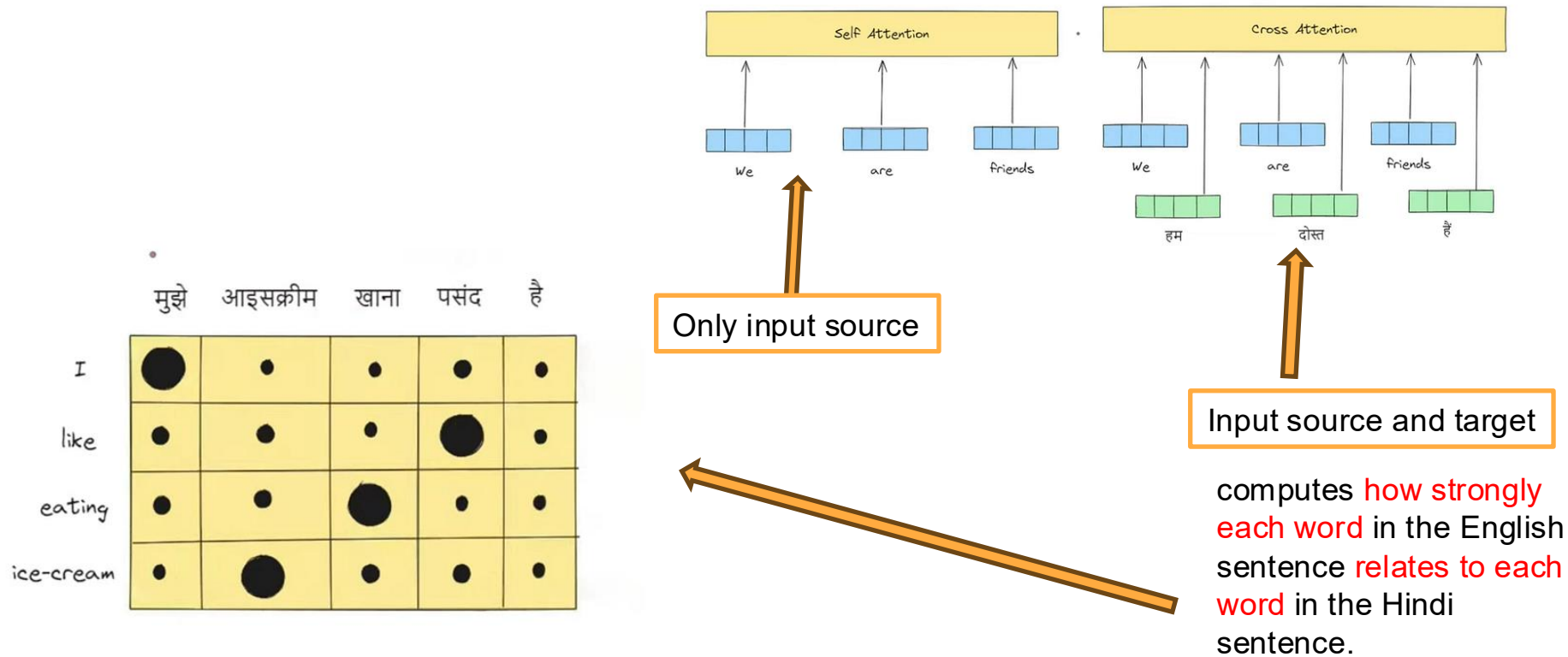


causal + padding Masking

Encoder-Decoder- Cross attention



Encoder-Decoder- Cross attention



Encoder-Decoder- Cross attention

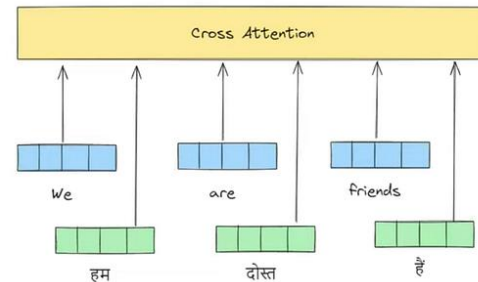
Decoder produces intermediate states Y

Encoder produces source representations X

(B, N_y, D) (B, N_x, D)

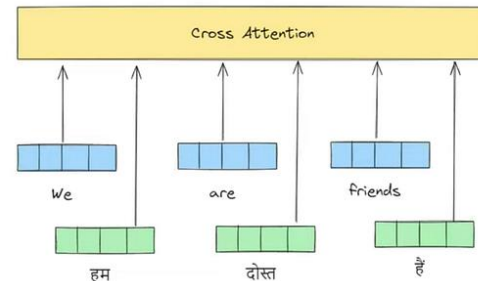
$$Q = YW_Q, \quad K = XW_K, \quad V = XW_V$$

$$\text{CrossAttn}(Y, X) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M \right) V \Rightarrow (B, H, N_y, N_x)$$



Encoder-Decoder- Cross attention

- **Machine Translation:** Translating text from one language to another involves comparing and aligning words between the source and target languages.
- **Question Answering:** Determining which parts of the context are relevant to answer a given question.
- **Image Captioning:** Generating descriptive text for images, where the image and text are treated as different modalities.
- **Text-to-Image Generation:** Creating images based on textual descriptions.
- **Text-to-Speech:** Generating speech from text, where the input is text and the output is speech.

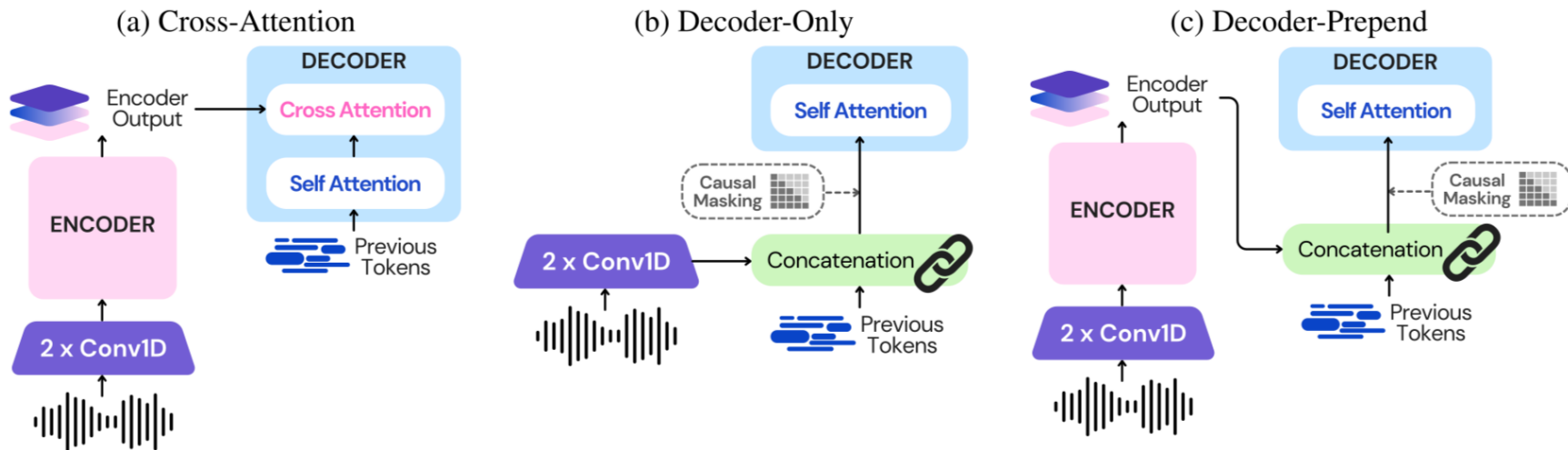


Encoder-Decoder

Model family	Self-attention	Cross-attention	Masks used	Positional encoding (common)
Encoder-only (BERT-like)	Bidirectional	No	Padding mask	Learned absolute / relative (varies)
Decoder-only (GPT-like)	Causal (look-ahead)	No	Causal + padding	RoPE (common)
Encoder-decoder (T5/seq2seq)	Encoder: bidirectional Decoder: causal	Yes (decoder → encoder)	Encoder: pad Decoder: causal + pad Cross-attn: encoder padding	RoPE or other relative methods

Encoder-Decoder

Speech to text



Next steps

- Training LLM !

Additional Material

<https://poloclub.github.io/transformer-explainer/>
[youtube.com/watch?v=mmzRYGCfTzc&themeRefresh=1](https://www.youtube.com/watch?v=mmzRYGCfTzc&themeRefresh=1)

Thank you!

If you want to ask us any questions about the module, the university, or anything else, drop us an email.

Pro tip: write **COMM117** on the email subject line, to make sure we give it priority.



r.mu2@exeter.ac.uk

Questions?